ModelFlow - Model Specification

August 13, 2026

Contents

0.1 Introduction

This supplement is about **how a model is specified in ModelFlow** — the step that turns a piece of readable text into a model object that can be simulated on a Pandas DataFrame.

A ModelFlow model is written in a small domain specific language (DSL). Each equation states a piece of business logic, one endogenous variable per equation:

```
> profit__{banks} = revenue__{banks} - expenses__{banks}
```

Before such text can be solved, it has to be **expanded** (loops over lists, sums, tags), **normalized** (so the endogenous variable stands alone on the left-hand side) and optionally extended with **add factors**. `Makemodel` performs the expansion, `normal` performs the normalization, and the `%%Makemymodel` cell magic lets you do the whole thing in a notebook cell.

0.1.1 What is in this book

Model specification introduces the DSL and the `Makemodel` preprocessor: list definitions and `do`-loops, `doable`, `sum()`, the `<exo>`, `<add>` and `<fit>` tags, `replacements=` and `list_defs=`, and how several model blocks are combined with `+` into a `Listmodels` container.

The `%%Makemymodel` magic shows the notebook front end: writing Markdown documentation and `>` equations in the same cell, the available options (`show`, `draw`, `render`, `latex`), and how `segment=` lets you build a large model from independently developed pieces.

Normalization goes one level down, to the `normal` function that rewrites an equation into `y = F(y, x)` form, and to the `Normalized_frml` class that carries the original, preprocessed, normalized and fitted versions of an equation.

Add factors looks in detail at what the `<add>` tag actually generates, and how the resulting add factor can be used to control an equation.

0.1.2 Prerequisites

The notebooks import from the ModelFlow source, mainly:

```
from modelconstruct_estimation import Makemodel
import modeljupytermagic          # registers the %%Makemymodel magic
from modelnormalize import normal
```

Each notebook can be read on its own, but the chapters are ordered so that later ones build on the vocabulary of the earlier ones.

0.2 Modelflow brings together a model and a Pandas DataFrame

ModelFlow is designed to combine a **model specification** with a **Pandas DataFrame** that contains the corresponding data.
In this setup, each **column** of the DataFrame represents a model variable, while the **row index** represents time.
This structure allows ModelFlow to evaluate and simulate the model directly on the data, period by period.

While Pandas provides powerful tools for managing time series, it does not natively handle **lagged variables** or **simultaneous equation systems** in a convenient way.
ModelFlow fills this gap by extending the DataFrame framework with capabilities for **lag and lead referencing**, **recursive and simultaneous solution algorithms**, and **automatic dependency handling** between variables.

In this way, ModelFlow bridges the gap between **data storage** and **model computation**, making it possible to define, solve, and analyze complex dynamic systems directly within the familiar Python and Pandas environment.

```
from modelconstruct_estimation import Makemodel
import modeljupytermagic
```

0.2.1 Specification of a model

In its simplest form, the ModelFlow domain-specific language (DSL) consists of **formulas**, each defining a relationship between variables.
However, ModelFlow also supports a **higher-level DSL** that extends this basic syntax with Python-based text processing features.
This enables the user to **generate formulas programmatically**, allowing the model specification itself to be written at a **higher level of abstraction**.

With this extended DSL, users can define **templates or patterns** for equations that apply across multiple entities — for example, all sectors in a macroeconomic model or all banks in a stress-testing framework — and then automatically expand them into a complete set of detailed, entity-specific equations.

This approach dramatically reduces **boilerplate text** and improves **consistency** across the model. Instead of manually writing large blocks of repetitive equations, the modeler can rely on Python constructs such as loops, lists, and string formatting to generate the full specification dynamically and transparently.

By integrating this higher-level DSL directly into the modeling framework, ModelFlow combines the **clarity and expressiveness** of a concise formula language with the **flexibility and automation** capabilities of Python — making it possible to specify large, structured models in a compact, readable, and reproducible way.

0.2.2 Example Models

To illustrate the use of ModelFlow and its formula-based DSL, we present two simple examples.
The first is a small macroeconomic model describing the relationship between **GDP** and **consumption**.
The second is a **debt dynamics** model that tracks the evolution of government debt over time.
Together, they demonstrate how behavioral and accounting identities can be expressed concisely in ModelFlow's syntax.

1. GDP and Consumption Model

This simple model links output Y_t and consumption C_t over time.
Consumption depends on lagged disposable income, and output is determined by aggregate demand.

$$C_t = \alpha_0 + \alpha_1 Y_t + \alpha_2 Y_{t-1} + \varepsilon_t^C$$
$$Y_t = C_t + I_t + G_t + X_t - M_t$$

- C_t – consumption at time t
- Y_t – output (GDP) at time t
- I_t, G_t, X_t, M_t – investment, government spending, exports, and imports
- α_0, α_1 , α_2 – behavioral parameters
- ε_t^C – consumption shock

This captures **lagged adjustment behavior** (consumption depends on past income) while treating the other demand components as **exogenous**. Also the model is contains **contemporaneous feedback loops** so it has to be solved as a system.

To input this into ModelFlow the two formulas can be written as:

```
Frml <stoc> C = a0 +  a1*Y + a2*Y( -1) + epsC $
Frml Y = C + I + G + X - M $
```

---

2. Debt Dynamics Model

A second example tracks the evolution of government debt over time.
Debt increases with the primary deficit and with interest payments on existing debt.

$$D_t = (1 + i_{t-1})D_{t-1} + PD_t$$
$$PD_t = G_t - T_t$$

- D_t – government debt at time t
- i_{t-1} – interest rate on debt from the previous period
- PD_t – primary deficit (spending minus taxes)
- G_t – government spending
- T_t – taxes

Here, D_t is **endogenous**, while i_t, G_t, and T_t may be treated as **exogenous inputs** or determined by other model blocks.

This simple dynamic ensures that current debt depends on **lagged debt** and **current fiscal flows**, a common structure in fiscal and macroeconomic models.

To input this into ModelFlow the two formulas can be written as:

```
Frml D  = (1 + i( -1))*D( -1) + PD $
Frml PD = G - T $
```

0.2.3 A Model

For the purpose of this note, a (simple) model like the two mentioned above can be thought of as a system of equations that links **endogenous** and **exogenous** variables over time.

- **Endogenous variables** are those determined *within* the model — their values are the outcome of the relationships the model describes.

- **Exogenous variables**, on the other hand, are determined *outside* the model — they are treated as given inputs or shocks that influence the endogenous variables but are not themselves affected by them.

In time-based models, variables can appear with **lags** or **leads**.
A *lag* refers to the value of a variable in a previous period — for example, y_{t-1} is the value of y one period before t.
A *lead* refers to a future value — for example, y_{t+1} represents the value of y one period after t.

Including lags allows the model to capture persistence and delayed effects in economic relationships, such as habit formation or gradual adjustment.
In this note, only **lags** will be used, as they are sufficient for representing the backward-looking relationships typical of most empirical and accounting-based models.
However, **ModelFlow** also allows the use of *leaded* variables, which are useful in models with rational or model-consistent expectations.

Lets say we have:

- **n**: number of endogenous variables
- **k**: number of exogenous variables
- **r**: maximum lag of endogenous variables
- **s**: maximum lag of exogenous variables
- **t**: time index (year, quarter, month, day, or any other unit)

$$
\begin{aligned}
y_t^1 &= f^1(\, y_t^2, \ldots, y_t^n \;,\; y_{t-1}^1, \ldots, y_{t-r}^n \;,\; x_t^1, \ldots, x_t^k \;,\; x_{t-1}^1, \ldots, x_{t-s}^k \,) \\
y_t^2 &= f^2(\, y_t^1, \ldots, y_t^n \;,\; y_{t-1}^1, \ldots, y_{t-r}^n \;,\; x_t^1, \ldots, x_t^k \;,\; x_{t-1}^1, \ldots, x_{t-s}^k \,) \\
&\vdots \\
y_t^n &= f^n(\, \underbrace{\underbrace{y_t^1, \ldots, y_t^{n-1}}_{\text{current}}, \underbrace{y_{t-1}^1, \ldots, y_{t-r}^n}_{\text{lagged}}}_{\text{endogenous}}, \underbrace{\underbrace{x_t^1, \ldots, x_t^k}_{\text{current}}, \underbrace{x_{t-1}^1, \ldots, x_{t-s}^k}_{\text{lagged}}}_{\text{exogenous}} \,)
\end{aligned}
$$

Sometime such models are implemented directly in a programming environment such as Python, Matlab, or even Excel, where they are both *specified* and *solved* in the same code.

0.2.4 Why Separate Specification and Implementation?

Instead of mixing economic reasoning with programming details, a model can be written in a **domain-specific language (DSL)** designed specifically for economics and finance. Separating specification from implementation has several advantages:

1. **Clarity and focus** – The specification remains close to the underlying economic logic, expressed in a concise and readable form. The modeler concentrates on *what the relationships are*, not on *how the computer solves them.*

2. **Expressiveness** – The DSL uses syntax familiar to economists (equations, lags, variables), avoiding the clutter of general-purpose programming constructs.

3. **Maintainability** – Models can be extended or updated without rewriting solution code. Changing an equation is as simple as editing one line in the specification.

4. **Portability** – Once specified, the same model can be executed with different numerical engines or platforms without altering its economic content.

5. **Transparency** – The separation makes models easier to review, validate, and communicate, since the specification reads like economic theory rather than computer code.

This approach is common in several econometric systems such as **EViews**, **Dynare**, and **GEKKO**, which all use their own DSLs to describe models.

ModelFlow follows a similar philosophy but with a key difference: it does not introduce a new language for data handling, visualization, or model analysis. Instead, ModelFlow relies on the **Python ecosystem** - primarily `pandas`, but also `networkx`, `sympy`, `numpy`, and other libraries - allowing the model specification to be combined seamlessly with powerful tools for data manipulation, result analysis, and visualization.

0.2.5 A Simple Specification Language

In practice, the DSL for economic models is built around equation-like statements. For example:

```
Frml <stoc> C = a0 + a1*Y( -1) + epsC $
Frml Y = C + I + G + X - M $
```

Here each formula (`Frml`) defines the relationship between variables in a way that mirrors how economists usually write them, with lags (`(-1)`) and simple algebraic notation.

The optional `<tag>` is used for transforming the frml or to control solving - more on that later.

This kind of business-logic-oriented specification acts as the "blueprint" of the model, leaving the task of solving it to the underlying implementation engine.

0.2.6 Makemodel – Expanding the Stripped-Down DSL

The basic `Frml` statements form the **lowest level** of the ModelFlow domain-specific language (DSL), defining the direct relationships between variables. The basic `Frml` statements are transpiled into Python when the model is solved.

To make the language more expressive, concise, and suitable for large, structured models, ModelFlow uses a preprocessor called `Makemodel` to extends this foundation with several higher-level constructs:

- **Lists and sublists** – to organize entities such as banks, sectors, or countries, and automatically expand equations across these groups.
- **Do ... enddo loops** – to generate sets of equations programmatically over the members of a list or over a specified range.
- **Sum operator** – to express aggregation directly within equations by summing over lists or sublists.
- **Doable equations** – shorthand notation combining loops and summation, simplifying the definition of repeated or aggregated relationships.
- **Stripped-down syntax** – allows equations to be written without explicit `Frml` and `$` markers, resulting in cleaner, more compact specifications.
- **Model specifications embedded in Markdown** – each line of model code starts with `>`, making them visually distinct and easily readable within documents. Documentation and explanations can be formatted nicely alongside the code using Markdown.

Together, these features make it possible to write models at a **higher level of abstraction**, reducing repetition and improving consistency across model components.

Instead of defining each equation manually, the modeler can use lists, loops, and operators to express economic logic once and have ModelFlow automatically generate all the necessary detailed equations. This design allows the DSL to scale from small illustrative examples to large, multi-entity macroeconomic and financial models while maintaining clarity and transparency.

Makemodel Properties

Box 1 Makemodel Properties

For use by the very interested modeller, the `Makemodel` dataclass stores the full transformation state of a model as it moves from the stripped-down DSL to the fully expanded, normalized set of `Frml` statements ready for ModelFlow.
Each field corresponds to a specific stage in this expansion and normalization process.

Property	Type	Description
`original_statements`	`str`	Input model specification written in the stripped-down DSL.
`normal_frml`	`str`	Output containing the normalized model equations.
`normal_main`	`str`	Normalized `Frml` statements for the main model equations.
`normal_fit`	`str`	Normalized `Frml` statements for fitted-value calculations.
`normal_calc_add`	`str`	Normalized `Frml` statements for add-factor calculations.
`clean_frml_statements`	`str`	Intermediate version with properly formatted `Frml` lines and structured list syntax.
`post_doable`	`str`	Model after expansion of all `doable` equations.
`post_do`	`str`	Model after expansion of explicit `do ... enddo` loops.
`post_sum`	`str`	Model after expansion of all summation (`sum`) operators.
`expanded_frml`	`str`	Fully expanded set of executable `Frml` equations.
`normal_expressions`	`List[Any]`	List of parsed normalized expression objects.
`list_specification`	`str`	String representation of all list definitions found in the model.
`modellist`	`str`	Dictionary-like string of defined lists and their attributes.
`funks`	`List[Any]`	List of user-defined functions that can be referenced in model formulas.

These internal fields allow the modeller to follow, in detail, how the stripped-down DSL is gradually transformed into executable ModelFlow syntax.
They provide a complete diagnostic view of the intermediate stages—useful for debugging, educational purposes, or extending ModelFlow's preprocessing pipeline.

The higher-level properties such as **`show`** and **`mmodel`**—described in the main `Makemodel` section—build on this internal structure to present the final expanded model or to produce a fully functional ModelFlow model instance.

Lists

Among the higher-level constructs, **lists** and **sublists** are central to making ModelFlow scalable and expressive.

They allow the modeler to define **groups of entities** (such as banks, portfolios, sectors, or countries) and to attach **attributes** or **hierarchies** to those entities.
This enables equations to be written once as templates and then **expanded automatically** across all relevant list members.

Sublists can be used to define logical relationships between entities — for example, linking each bank to its home country, or associating portfolios with regulatory categories.
These structures make it possible to organize models systematically and to maintain consistency across entities that share the same behavioral equations.

This approach eliminates redundancy and manual repetition: instead of writing dozens or hundreds of similar equations, the modeler defines a **single equation pattern**, and ModelFlow expands it dynamically over the members of a list.

This capability is especially valuable in:

- **Stress-testing models** – where many banks, each with multiple portfolios, must be represented consistently.
- **Macroeconomic models** – where sectors of the economy (households, firms, government, rest of the world, etc.) share common behavioral structures.
- **Cross-country or panel models** – where the same specification is applied across multiple countries or regions.

Syntax List definitions in the ModelFlow DSL follow a simple and consistent format:

- Each list **starts** with the keyword `list`.
- It can contain one or more **sublists**, separated by `/`.
- Each sublist **starts** with a `name:` followed by its members.

Below are a few examples of list definitions:

```
lists ='''
## Lists used in the next examples
The folloving lists are only examples

First a list of banks and of the home country for each bank
>list BANKS = BANKS    : Danske  Nordea Ibs /
>             bankcountry : denmark sweden denmark  /
>             include   :   0        1       1

A list of countries
>list country = country : denmark sweden

A list of different portefolios and of the regulary treatment
>list ports = ports  : households   NFC    RE  sov /
>              reg  : AIRB  AIRB   STA  STA


'''
```

Meaning

- `list BANKS = ...` defines a top-level list of banks with an **sublist** `bankcountry` that specifies where each bank belongs.
- `list country = ...` creates a country list.
- `list ports = ...` groups portfolios (`households`, `NFC`, `RE`, `sov`) and creates **sublists** with regulatory categories (`AIRB`, `STA`).

So the list statement starts with the keyword `list` and a `list name =`. Sublists are seperated by / and each starts with a `sublist name:`

It makes sense to give the first sublist the same name as the list name.

The benefits of lists:

- **Compactness** – groups can be declared once and reused throughout the model.
- **Flexibility** – adding or removing members requires only updating the list, not the equations.
- **Transparency** – clear structure makes it easy to see which entities and attributes the model covers.

By defining lists in this way, ModelFlow provides a structured layer between the economic logic and the generated equations, enabling users to manage complex multi-entity models efficiently and transparently.

Do ... Enddo Loops

The `do ... enddo` construct allows repetitive patterns of equations to be generated automatically. When used together with **lists**, it enables the modeler to create a complete set of equations for every member of a list without having to write each one manually.

A `do` block defines a **loop** over the members of a specified list (for example, all banks, sectors, or countries).
Within the block, the loop variable can be referenced directly inside formulas — ModelFlow will automatically substitute the appropriate member name during expansion.

This mechanism is essential for large-scale models where the same behavioral relationships apply to multiple entities.
It ensures **consistency**, reduces **repetition**, and makes model specifications more **maintainable** and **transparent**.

Syntax The syntax of a `do` block is straightforward:

- The loop *starts* with: **do** `list name` $\underbrace{\texttt{sublist name} = \texttt{value}}_{\text{optional}}$
- It *ends* with: **enddo**

One or more Frml statements — and possibly nested do ... enddo blocks — are written inside the loop.

Makemodel automatically expands each Frml statement for every member of the specified list that satisfies the optional filter.

Example If the Python variable lists already defines some model lists, it can be combined with an example model specification and passed to `Makemodel` for processing.

The `show` property can then be used to display the resulting DSL equations after expansion.

```
list_example = Makemodel(lists +
'''
## Loop over all banks
>do banks
Calculate profit in each bank
>     profit__{banks} = revenue__{banks} - expenses__{banks}
Terminate the loop
> enddo
''')
list_example.show
```

```
FRML <> PROFIT__DANSKE = REVENUE__DANSKE -EXPENSES__DANSKE $
FRML <> PROFIT__NORDEA = REVENUE__NORDEA -EXPENSES__NORDEA $
FRML <> PROFIT__IBS = REVENUE__IBS -EXPENSES__IBS $
```

`.showmd` Will render whole Markdown text

```
list_example.render
```

```
<IPython.core.display.Markdown object>
```

Conditional Loops

The do statement can also include conditions that restrict the loop to members of a list matching specific attribute values. This is written using the syntax do =.

For example, suppose the list BANKS includes a sublist bankcountry, then the following block will only expand for banks located in Denmark:

```
Makemodel(lists +
'''
>do banks bankcountry = denmark
>     profit__{banks} = revenue__{banks} - expenses__{banks}
> enddo
''').show

FRML <> PROFIT__DANSKE = REVENUE__DANSKE -EXPENSES__DANSKE $
FRML <> PROFIT__IBS = REVENUE__IBS -EXPENSES__IBS $
```

ModelFlow will expand this block only for banks whose bankcountry attribute equals denmark.

This conditional looping makes it possible to write targeted relationships for specific subsets of entities for instance, defining different equations for domestic and foreign banks, or applying alternative behavioral rules to different sectors or regions.

By combining lists, sublists, and conditional do loops, ModelFlow allows highly structured models to be specified compactly while preserving clarity and flexibility.

Doable Equations

While `do ... enddo` loops provide an explicit way to expand formulas across members of a list, the **`doable`** construct offers a more concise and automatic alternative.

A **doable equation** is a *high-level shorthand* that combines the functionality of loops and summations without requiring the user to write repetitive loop structures.
Instead of manually specifying `do` blocks, ModelFlow can **detect the relevant lists automatically** by inspecting the **first** variable names in the equation.

Variables that contain double underscores (`__`) indicate **list dimensions**.
For example, in `profit__{banks}`, ModelFlow recognizes that `profit` is a variable defined over the list `BANKS`.
Similarly, in `loss__{banks}__{ports}`, the two underscores identify two list dimensions — `BANKS` and `PORTS`.

Box 2 Doable summary

$$\texttt{doable}\ \underbrace{\langle tag\rangle}_{\text{Tag}}\ \underbrace{[list = sublist = value, \ldots]}_{\text{Optional Conditions}}\ \underbrace{\underbrace{\texttt{VARIABLE}}_{\text{LHS Expression}}\ \underbrace{\_\_\{list1\}\_\_\{list2\}\ldots}_{\text{LHS Dimensions}}}_{\text{LHS}} = \underbrace{\texttt{Expression including subl}}_{\text{RHS Expression}} \tag{1}$$

When ModelFlow encounters a `doable` equation such as:

```
Makemodel(lists + '''
>doable profit__{banks} = revenue_{banks} - expenses_{banks}
''').show

FRML <> PROFIT__DANSKE = REVENUE_DANSKE -EXPENSES_DANSKE $
FRML <> PROFIT__NORDEA = REVENUE_NORDEA -EXPENSES_NORDEA $
FRML <> PROFIT__IBS = REVENUE_IBS -EXPENSES_IBS $
```

it automatically expands the formula across all members of the list BANKS — just as a do BANKS ... enddo loop would.

Thus, the doable keyword eliminates the need for explicit looping syntax, making the specification both shorter and easier to read.

Doable equations can also include conditions and tags in the same way as do loops. For example:

```
Makemodel(lists+'>doable [banks= bankcountry=denmark] profit__{banks}=revenue__{banks} -expens

FRML <> PROFIT__DANSKE = REVENUE__DANSKE -EXPENSES__DANSKE $
FRML <> PROFIT__IBS = REVENUE__IBS -EXPENSES__IBS $
```

Expands only for banks where the sublist bankcountry equals denmark.

```
Makemodel(lists+'>doable [banks=include] profit__{banks}=revenue__{banks} -expenses__{banks}')

FRML <> PROFIT__NORDEA = REVENUE__NORDEA -EXPENSES__NORDEA $
FRML <> PROFIT__IBS = REVENUE__IBS -EXPENSES__IBS $
```

Expands only for banks where the sublist include has the value 1

This mechanism is particularly powerful when dealing with complex, multi-dimensional systems — such as stress-test models with many banks and portfolios — since the modeler can write compact, pattern-based equations that ModelFlow expands automatically by detecting all relevant list dimensions.

In short, doable equations serve as an elegant, high-level syntax that:

- Avoids the boilerplate of do ... enddo structures,
- Automatically determines which lists apply by analyzi ng variable dimensions, and
- Keeps model specifications concise, readable, and scalable.

0.2.7 Sum of variables

To sum over lists the sum statement en expressions can be used. it will generate a sum over the list and potential filtered by a condition

```
Makemodel(lists+'>loss_danske =  sum(ports,loss__danske__{ports})').show

FRML <> LOSS_DANSKE = (LOSS__DANSKE__HOUSEHOLDS+LOSS__DANSKE__NFC+LOSS__DANSKE__RE+LOSS__DANSK
```

Nested sums

```
Makemodel(lists+'>loss_total =  sum(banks,sum(ports,loss__{banks}__{ports}))').show

FRML <> LOSS_TOTAL = ((LOSS__DANSKE__HOUSEHOLDS+LOSS__DANSKE__NFC+LOSS__DANSKE__RE+LOSS__DANSK
```

Conditional sum

Here the total sum for all banks for portefolios with reg=sta

```
Makemodel(lists+ '>loss_total_sta =  sum(banks,sum(ports reg=STA,loss__{banks}__{ports}))').sh

FRML <> LOSS_TOTAL_STA = ((LOSS__DANSKE__RE+LOSS__DANSKE__SOV)+(LOSS__NORDEA__RE+LOSS__NORDEA_

Makemodel(lists+ '>loss_total_sta =  sum(banks include,sum(ports reg=STA,loss__{banks}__{ports

FRML <> LOSS_TOTAL_STA = ((LOSS__NORDEA__RE+LOSS__NORDEA__SOV)+(LOSS__IBS__RE+LOSS__IBS__SOV))
```

0.2.8 Combining doable and sum

In `doable` to create a sum of all the calculated Conditional the tag sum can be used. This will generate a sum of all the calculated values in the statement. `<sum=text>` will give dimension name to the result. If no text is given it will be called sum.

```
Makemodel(lists+
'>doable <sum=total>  profit__{banks}=revenue__{banks} -expenses__{banks}'
        ).show
```

```
FRML <SUM=TOTAL> PROFIT__DANSKE = REVENUE__DANSKE -EXPENSES__DANSKE $
FRML <SUM=TOTAL> PROFIT__NORDEA = REVENUE__NORDEA -EXPENSES__NORDEA $
FRML <SUM=TOTAL> PROFIT__IBS = REVENUE__IBS -EXPENSES__IBS $
FRML <SUM=TOTAL> PROFIT__TOTAL = (PROFIT__DANSKE+PROFIT__NORDEA+PROFIT__IBS) $
```

And with conditions it works to

```
exploded = Makemodel(lists+
'>doable <sum=total> [banks bankcountry=denmark] profit__{banks}=revenue__{banks} -expenses__{
        )
exploded.show
```

```
FRML <SUM=TOTAL> PROFIT__DANSKE = REVENUE__DANSKE -EXPENSES__DANSKE $
FRML <SUM=TOTAL> PROFIT__NORDEA = REVENUE__NORDEA -EXPENSES__NORDEA $
FRML <SUM=TOTAL> PROFIT__IBS = REVENUE__IBS -EXPENSES__IBS $
FRML <SUM=TOTAL> PROFIT__TOTAL = (PROFIT__DANSKE+PROFIT__NORDEA+PROFIT__IBS) $
```

`.draw` — Displaying the Model's Causal Structure

The **`.draw`** method visualizes the **causal structure** of a model as defined by its equations.
It provides an intuitive graph of dependencies between variables, helping to understand the logical flow and interconnections within the model.

> **Warning**
> Use this feature only for **small models** — for larger models, the resulting diagram can become cluttered.

```
exploded.draw
```

The **`Makemodel.draw`** property is a convenient wrapper around **`Makemodel.mmodel.drawmodel()`**. It first accesses the **`modelclass.model`** instance defined by the DSL, then generates a graphical representation of the entire model.

All methods available to the underlying model instance can be used — for example, the **`.tracedep`** method, which allows tracing the dependencies of a single variables within the model.

```
exploded.mmodel.PROFIT__DANSKE.tracepre()
```

0.2.9 Tags and Formula Decoration

As mentioned earlier, the user should focus primarily on defining the **economic relationships** themselves.
However, in some cases it can be useful to **decorate** formulas to enable specific **modifications or behaviors** during calculations.
This is achieved by **tagging** the formula.

If we start with this simple formula and no tags. The formula will just be translated to a simple DSL formula:

```
Makemodel('>a = b').show
```

```
FRML <> A = B $
```

Enable fixing the value of a a variable

When the value of a variable is known for certain time periods, it can be fixed by tagging its formula with exo. This is especially useful in forecasting exercises, where newly published statistics are available and should override the model's computed values.

When a formula is tagged with exo, two additional variables are automatically created:

- _X — the fixed (exogenous) value to be used.
- _D — a dummy variable that equals 1 in the periods where the fixed value should replace the model-generated one.

```
Makemodel('><exo> a = b').show

FRML <EXO> A = (B)* (1 -A_D)+ A_X*A_D $
```

his means that:

- When `A_D = 0`, the model uses the endogenous calculation A = B.
- When `A_D = 1`, the model instead uses the fixed value A = A_X.

This mechanism allows the modeller to incorporate known or published values seamlessly, without modifying the model's structure.

The ModelFlow model instance also includes convenient helper methods for managing these exogenous values — for example, to set, update, or release fixed variables during forecasting or simulation exercises.

Include add factors

In some it can be useful to be able to adjust the formula in order to make an experiment or to incorporate factors outside the model. Sometime called **add factors** or **expert judgements** To make life easy this can be done by using the tag `add` also the tag `add_suffic=` can be used to change the suffix for the add factor from the default `_A`

```
Makemodel('><add> a = b').show

FRML <ADD> A = (B + A_A)                                    $

FRML <CALC_ADD_FACTOR>  A_A = (A) - (B) $
```

Reverse Calculation of Add Factors

A special `Frml` tagged with `CALC_ADD_FACTOR` is automatically generated by **Makemodel**.
This formula is used by the **ModelFlow model instance** to build a specialized *calculation model* that computes the **add factors** — the residual adjustments needed to reconcile actual or fixed values with the model's baseline equations.

On its own, this feature is not particularly useful, but when **combined with the `exo` tag**, it becomes powerful.
Together, they allow the model to **retain its marginal (behavioral) properties** even when some variables have **fixed values**.

After the add factors have been computed, the modeller can **reset the corresponding `_D` dummies to 0**, effectively releasing the fixed variables.
The model can then be used for **experiments, policy simulations, or shock analyses**, while the previously calculated add factors ensure internal consistency and preserve the model's original fit to observed data.

```
Makemodel('><exo,add> a = b').show

FRML <EXO,ADD> A = (B + A_A)* (1 -A_D)+ A_X*A_D  $

FRML <CALC_ADD_FACTOR>  A_A = (A) - (B) $
```

Naming of Add Factors

If the modeller prefers a **different naming convention** for add factors, this can be achieved by including the tag **add_suffix=** in the formula specification.
For example:

```
Makemodel('><add,add_suffix=__J> a = b').show

FRML <ADD,ADD_SUFFIX=__J> A = (B + A__J)                                    $

FRML <CALC_ADD_FACTOR>  A__J = (A) - (B) $
```

generate add factors named with the suffix _J instead of the default _A.

Including Calculated Fitted Values

When **add factors** or **fixed values** are used, it may be useful to compute the value implied by the **original fitted formula** — that is, the model's unadjusted behavioral relationship.
This can be achieved by tagging the formula with **FIT**.

When the FIT tag is applied, an additional formula is automatically created.
This formula calculates a new variable with the suffix **_FITTED**, representing the value generated by the **original model equation** (before the inclusion of add factors or fixed values).

This is helpful for comparing the model's baseline predictions with adjusted or constrained outcomes, especially in forecasting and diagnostic analysis.

```
Makemodel('><exo,add,fit> a = b').show

FRML <EXO,ADD,FIT> A = (B + A_A)* (1 -A_D)+ A_X*A_D  $
FRML <FIT>  A_FITTED = B $
FRML <CALC_ADD_FACTOR>  A_A = (A) - (B) $

Makemodel(lists+
'>doable <sum=total,exo,fit,add>  profit__{banks}=revenue__{banks} -expenses__{banks}'
        ).show

FRML <SUM=TOTAL,EXO,FIT,ADD> PROFIT__DANSKE = (REVENUE__DANSKE -EXPENSES__DANSKE + PROFIT__DAN
FRML <SUM=TOTAL,EXO,FIT,ADD> PROFIT__NORDEA = (REVENUE__NORDEA -EXPENSES__NORDEA + PROFIT__NOR
FRML <SUM=TOTAL,EXO,FIT,ADD> PROFIT__IBS = (REVENUE__IBS -EXPENSES__IBS + PROFIT__IBS_A)* (1 -
FRML <SUM=TOTAL,EXO,FIT,ADD> PROFIT__TOTAL = ((PROFIT__DANSKE+PROFIT__NORDEA+PROFIT__IBS) + PR
FRML <FIT>  PROFIT__DANSKE_FITTED = REVENUE__DANSKE -EXPENSES__DANSKE $
FRML <FIT>  PROFIT__NORDEA_FITTED = REVENUE__NORDEA -EXPENSES__NORDEA $
FRML <FIT>  PROFIT__IBS_FITTED = REVENUE__IBS -EXPENSES__IBS $
FRML <FIT>  PROFIT__TOTAL_FITTED = (PROFIT__DANSKE+PROFIT__NORDEA+PROFIT__IBS) $
FRML <CALC_ADD_FACTOR>  PROFIT__DANSKE_A = (PROFIT__DANSKE) - (REVENUE__DANSKE -EXPENSES__DANS
FRML <CALC_ADD_FACTOR>  PROFIT__NORDEA_A = (PROFIT__NORDEA) - (REVENUE__NORDEA -EXPENSES__NORD
FRML <CALC_ADD_FACTOR>  PROFIT__IBS_A = (PROFIT__IBS) - (REVENUE__IBS -EXPENSES__IBS) $
FRML <CALC_ADD_FACTOR>  PROFIT__TOTAL_A = (PROFIT__TOTAL) - ((PROFIT__DANSKE+PROFIT__NORDEA+PR
```

0.2.10 Using replacements= or str.replace() to simplify multi-dimensional formulas

When a formula includes several dimensions (e.g., banks, country, ports), variable names can become long and repetitive.
To keep model definitions compact and readable, you can define formulas with **placeholders** (such as __dim) and then insert the actual dimension names automatically.

There are now **two ways** to do this:

1. Use Python's built-in `str.replace()` directly in the formula string.
2. Use the **`replacements=` argument** in `Makemodel`, which applies the same logic internally‘.

The replacements argument accepts either **a single (pattern, replacement) tuple** or **a list of such tuples** for multiple substitutions.

Why this helps

- **Cleaner input:** Focus on the formula's logic instead of long dimension names.
- **Consistency:** Reuse the same placeholder pattern across multiple equations.
- **Maintainability:** Change dimensional naming once in the `replacements` list instead of editing all formulas.
- **Convenience:** The `replacements=` argument avoids having to call `.replace()` manually.

Example

Below, the same model is defined in two equivalent ways — first using `.replace()`, and then using the `replacements=` argument.

```
# Example 1  Using str.replace() manually
Makemodel(lists +
    '>doable <> loss__dim = portefolio__dim * pd__dim * lgd__dim'
        .replace('__dim', '__{banks}__{country}__{ports}')
).show

repl=('__dim', '__{banks}__{country}__{ports}')

Makemodel(lists+
'>doable <>  loss__dim = portefolio__dim * pd__dim*lgd__dim',
replacements=repl      ).show
```

0.2.11 Using `list_defs=` to define lists in `Makemodel`

In many cases, it can be convenient to provide **list definitions** as an argument to `Makemodel` rather than embedding them directly into the model specification string.

This approach helps separate the **technical structure** (lists, dimensions, replacements) from the **economic logic** (the actual formulas), making your model files cleaner and easier to read.

When to use `list_defs=`

- When several submodels share the **same list definitions**.
- When you want to keep the **economic equations visible**.

How it works

You can pass a string containing the list definitions directly to `Makemodel` using the `list_defs=` argument.
`Makemodel` automatically prepends these lists to the model specification before parsing and normalization.

```
Makemodel(lists +'>doable profit__{banks} = revenue_{banks} - expenses_{banks}').show

FRML <> PROFIT__DANSKE = REVENUE_DANSKE -EXPENSES_DANSKE $
FRML <> PROFIT__NORDEA = REVENUE_NORDEA -EXPENSES_NORDEA $
FRML <> PROFIT__IBS = REVENUE_IBS -EXPENSES_IBS $

Makemodel('>doable profit__{banks} = revenue_{banks} - expenses_{banks}',list_defs=lists).show

FRML <> PROFIT__DANSKE = REVENUE_DANSKE -EXPENSES_DANSKE $
FRML <> PROFIT__NORDEA = REVENUE_NORDEA -EXPENSES_NORDEA $
FRML <> PROFIT__IBS = REVENUE_IBS -EXPENSES_IBS $
```

0.2.12 Using `functools.partial` to create simplified `Makemodel` constructors

When building many model components that share the same configuration — such as common `list_defs`, `replacements`, or `funks` —
you can simplify your workflow by creating a **pre-configured version** of `Makemodel` using `functools.partial`.

This avoids repeating the same arguments for every model block and makes your code cleaner and more consistent.

Why this helps

- **Less repetition:** No need to retype the same `list_defs` or `replacements` each time.
- **Consistency:** All model blocks use identical definitions and structural setup.
- **Convenience:** The resulting constructor behaves exactly like `Makemodel`, but with defaults already filled in.

Example:

0.2.13 Building modular models by combining smaller formula groups

Complex models often consist of multiple logical components — such as behavioral equations, accounting identities, or sector-specific submodels.
Instead of defining one large block of equations in a single string, it's often more efficient to **specify smaller groups of formulas** and then combine them into a complete model using the `+` operator.

Why this approach is beneficial

- **Modularity:** Each `Makemodel` object can represent a distinct part of the model — for example, a banking block, a household sector, or a price system.
- **Clarity:** Smaller, self-contained equation groups are easier to read, debug, and test independently.
- **Reusability:** You can reuse model fragments across projects or scenarios by simply combining them differently.
- **Incremental development:** Start with a minimal subset of formulas, verify behavior, and then expand the model progressively.

How it works

The + operator merges multiple `Makemodel` objects into a single `Listmodels` container, which behaves like a unified model.
This aggregation does not recompute anything — it simply combines their normalized forms, making it fast and memory-efficient.

Below, we define two small model blocks:

Each block is defined as an independent `Makemodel`, and then we combine them using +.
This creates a single integrated model (`Listmodels`) that can be displayed or solved as one unit.

In larger models, it's good practice to define separate logical components (blocks) for different economic or financial relationships.
Here, we define three related parts of a simple very stylized credit risk model:

1. **PD update block** – describes how each bank's probability of default (`PD`) evolves.
2. **Expected Loss block** – calculates expected losses as a function of exposure, PD, and loss given default (`LGD`).
3. **Bank Expected Loss block** – calculates expected losses for each bank.

Each block is created as an independent `Makemodel` object.
We then combine them with the + operator to form a complete integrated model (`Listmodels`), which can be solved or analyzed as one structure.

```
from functools import partial
SMakemodel = partial(Makemodel,list_defs=lists, replacements=('__dim', '__{banks}__{country}__

SMakemodel('>doable <>  loss__dim = portefolio__dim * pd__dim*lgd__dim').show

# - - - Define three small, modular formula groups - - -
# first define the replacements
repl = ('__dim', '__{banks}__{country}__{ports}')
# - - - Block 1: PD update equations - - -
pd_update_block = Makemodel(lists+
'>doable <> pd__dim = 0.02 + 0.8 * pd__dim( -1) + 0.1 * gdp_growth__{country}',replacements=re

# - - - Block 2: Expected loss calculation - - -
expected_loss_block = Makemodel(lists +
'>doable <> el__dim = exposure__dim * pd__dim * lgd__dim',replacements=repl)

# - - - Block 3: Expected loss calculation per bank - - -
bank_block = Makemodel(lists +
    '''
>    do banks
>      Expected_loss_{banks} = sum(country, sum(ports, el__dim))
>    enddo
    '''
    ,replacements=repl)

# - - - Combine all blocks into one model - - -
credit_risk_model = sum([pd_update_block + expected_loss_block + bank_block])
credit_risk_model.show
```

Of course we could also just make the model in one Makemodel or even simpler a SMakemodel that was defined above.

```
total_credit_risk_model = SMakemodel('''
>doable <> pd__dim = 0.02 + 0.8 * pd__dim( -1) + 0.1 * gdp_growth__{country}
>doable <> el__dim = exposure__dim * pd__dim * lgd__dim
>do banks
>   Expected_loss_{banks} = sum(country, sum(ports, el__dim))
>enddo ''')
total_credit_risk_model.show
```

And the causal structure can be displayed

```
credit_risk_model.draw
```

0.2.14 %%Makemymodel — a Jupyter magic wrapper for Makemodel

Using Jupyter notebooks it can be useful to let the test, equations and lists be defines in a whole cell.

- `%%Makemymodel` is a Jupyter **cell magic** that creates a `Makemodel` instance from Markdown-based model text.
- `%Makemymodel` is the corresponding **line magic**, used to assemble a model from its segments.

```
%%Makemymodel <name> [options]
text ...
> equation
>> continuation
more text and equations
```

The cell content is used to create a `Makemodel` instance.

- If **segments are not used**, the instance is named `<name>`
- If **segments are used**, the instance is named after the current `<segment name>`

Segments

Segments allow you to experiment with parts of a model independently.
The full model can later be assembled using the line magic:

```
%Makemymodel <name> [options]
```

If

- `<segment name>` starts with `list`, the lists defined in the segment are used for all segments with the same `<name>`

Rules -

- `>` starts a model equation
- `»` continues the previous equation
- All other text is treated as Markdown

Common options

Option	Meaning
`render`	Render the model as Markdown
`show`	Print normalized equations
`draw`	Draw the dependency graph
`segment=<segment name>`	Store the cell as a named model segment

Summary

Markdown for humans, > for models.

```
%%Makemymodel test show
><add> a = log(b)
```

```
<IPython.core.display.Markdown object>
```

```
FRML <ADD> A = (log(B) + A_A)                                    $

FRML <CALC_ADD_FACTOR>  A_A = (A) - (log(B)) $
```

```
%%Makemymodel test show
><add> diff(a) = b
```

```
<IPython.core.display.Markdown object>
```

```
FRML <ADD> A = A_A+ (B) +A( -1) $

FRML <CALC_ADD_FACTOR>  A_A = A - ((B)) -A( -1) $
```

```
%%Makemymodel test show
><add> log(a) = b
```

```
<IPython.core.display.Markdown object>
```

```
FRML <ADD> A = EXP(A_A+ (B) ) $

FRML <CALC_ADD_FACTOR>  A_A = - ((B)) +LOG(A) $
```

```
%%Makemymodel test show
><add> dlog(a) = b
```

```
<IPython.core.display.Markdown object>
```

```
FRML <ADD> A = A( -1)*EXP(A_A+ (B) ) $

FRML <CALC_ADD_FACTOR>  A_A = - ((B)) +LOG(A) -LOG(A( -1)) $
```

```
%%Makemymodel test show
><add> log(a+c) = b
```

```
<IPython.core.display.Markdown object>
```

```
FRML <ADD> A = -C+EXP(A_A+ (B) ) $

FRML <CALC_ADD_FACTOR>  A_A = - ((B)) +LOG(A+C) $
```

```
%%Makemymodel test show
><add,implicit,endo=price> demand = supply
```

```
<IPython.core.display.Markdown object>
```

```
FRML <ADD,IMPLICIT,ENDO=PRICE> PRICE___RES = ( SUPPLY ) - ( DEMAND ) $
```

```
%%Makemymodel test show
><exo,add,fit> a = b
```

```
<IPython.core.display.Markdown object>
```

```
FRML <EXO,ADD,FIT> A = (B + A_A)* (1 -A_D)+ A_X*A_D  $
FRML <FIT>  A_FITTED = B $
FRML <CALC_ADD_FACTOR>  A_A = (A) - (B) $
```

0.3 The %%Makemymodel Jupyter Magic

The `%%Makemymodel` cell magic lets you write **model specifications directly in notebook cells** using Markdown-style text mixed with ModelFlow DSL equations. Under the hood it parses the cell, feeds everything to `Makemodel`, and pushes the resulting object into the notebook namespace — ready to use.

This notebook explains how it works and walks through all the options.

```
%load_ext autoreload
%autoreload 2

import modeljupytermagic          # registers the magics
from modelconstruct_estimation import Makemodel
```

0.3.1 1. Basic syntax

The first line after `%%Makemymodel` carries the **model name** and optional **flags**:

```
%%Makemymodel <name> [option1] [option2=value] ...
```

The rest of the cell is **Markdown text** where lines starting with `>` are treated as ModelFlow DSL equations. Everything else is documentation that gets rendered in the notebook.

After execution the `Makemodel` instance is available as a variable with the given `<name>`.

Minimal example

The cell below creates a `Makemodel` object called `simple` and renders the Markdown text in the notebook.

```
%%Makemymodel simple
A simple two -equation model
> y = c + i
> c = 0.8 * y( -1)
```

```
<IPython.core.display.Markdown object>
```

The variable `simple` is now a `Makemodel` instance in the notebook namespace. We can inspect it:

```
# Show the expanded equations
simple.show
```

```
FRML <> Y = C+I $
FRML <> C = 0.8*Y( -1) $
```

```
# Draw the causal graph
simple.draw
```

0.3.2 2. Available options

Options are placed on the first line after the model name. They follow the pattern `key` (boolean flag) or `key=value`.

Option	Default	Effect
`show`	off	Print the expanded equations after creation
`draw`	off	Display the causal dependency graph
`display`	off	Verbose mode: render Markdown **and** print equations + segment info
`render`	on	Render the Markdown text in the cell output
`render_list`	on	Include list definitions when rendering (set to `0` to hide them)
`latex`	off	Generate a LaTeX PDF of the model specification
`spec`	`markdown`	Rendering format (`markdown` by default)
`segment`	off	Segmented model building (see Section 4)
`replacements`	none	Dimension placeholders, e.g. `replacements=('__d','__{banks}')`
`funks`	none	User-defined functions to pass to `Makemodel`

Boolean options can be turned off explicitly with `option=0` or `option=False`.

Example: `show` and `draw`

```
%%Makemymodel demo show draw
A model with tags
><exo,add> y = c + i
> c = 0.8 * y( -1)

<IPython.core.display.Markdown object>

FRML <EXO,ADD> Y = (C+I + Y_A)* (1 -Y_D)+ Y_X*Y_D  $
FRML <> C = 0.8*Y( -1) $

FRML <CALC_ADD_FACTOR>  Y_A = (Y) - (C+I) $
```

Example: suppressing rendering with `render=0`

```
%%Makemymodel quiet render=0 show
This text will NOT appear in the output
> a = b + c

<IPython.core.display.Markdown object>

FRML <> A = B+C $
```

0.3.3 3. Using lists and `doable` inside the magic

The cell content is passed directly to `Makemodel`, so all DSL features — lists, `do...enddo`, `doable`, `sum()`, tags — work exactly as they would in a normal `Makemodel()` call.

```
%%Makemymodel bank_model show
## Credit risk model

Define the banks and portfolios:

>list banks  = banks : Danske Nordea
>list ports  = ports : Mortgage Corporate

Calculate expected loss per bank and portfolio, then aggregate:

>doable <sum=total> el__{banks}__{ports} = exposure__{banks}__{ports} * pd__{banks}__{ports}

<IPython.core.display.Markdown object>

FRML <SUM=TOTAL> EL__DANSKE__MORTGAGE = EXPOSURE__DANSKE__MORTGAGE*PD__DANSKE__MORTGAGE $
FRML <SUM=TOTAL> EL__DANSKE__CORPORATE = EXPOSURE__DANSKE__CORPORATE*PD__DANSKE__CORPORATE $
FRML <SUM=TOTAL> EL__NORDEA__MORTGAGE = EXPOSURE__NORDEA__MORTGAGE*PD__NORDEA__MORTGAGE $
FRML <SUM=TOTAL> EL__NORDEA__CORPORATE = EXPOSURE__NORDEA__CORPORATE*PD__NORDEA__CORPORATE $
FRML <SUM=TOTAL> EL__TOTAL = ((EL__DANSKE__MORTGAGE+EL__DANSKE__CORPORATE)+(EL__NORDEA__MORTGA
```

0.3.4 4. Segmented models

Large models can be split across **multiple cells** using the `segment=<name>` option. This is one of the most powerful features of the magic.

How it works:

1. Each cell with `segment=<name>` stores its content in a dictionary called `<modelname>_dict`.
2. Segments whose name starts with `list` or `text` are treated as definitions/documentation — they are stored but only rendered, not yet compiled.
3. When a non-list/non-text segment is encountered, the magic combines **all list segments** with the **current cell** and builds the `Makemodel` model.
4. A final call **without** `segment` combines **all** stored segments into the complete model.

This lets you build a model incrementally, cell by cell, with explanatory text between sections.

```
%%Makemymodel stress segment=listtests
## List definitions
>list banks  = banks : Alpha Beta
>list ports  = ports : Retail SME

<IPython.core.display.Markdown object>

%%Makemymodel stress segment=pd_block show
## PD dynamics
Probability of default follows an AR(1) process driven by GDP growth:
>doable <> pd__{banks}__{ports} = 0.02 + 0.8 * pd__{banks}__{ports}( -1) + 0.1 * gdp_growth

<IPython.core.display.Markdown object>

FRML <> PD__ALPHA__RETAIL = 0.02+0.8*PD__ALPHA__RETAIL( -1)+0.1*GDP_GROWTH $
FRML <> PD__ALPHA__SME = 0.02+0.8*PD__ALPHA__SME( -1)+0.1*GDP_GROWTH $
FRML <> PD__BETA__RETAIL = 0.02+0.8*PD__BETA__RETAIL( -1)+0.1*GDP_GROWTH $
FRML <> PD__BETA__SME = 0.02+0.8*PD__BETA__SME( -1)+0.1*GDP_GROWTH $
```

```
%%Makemymodel stress segment=loss_block show
## Expected loss
>doable <sum=total> el__{banks}__{ports} = exposure__{banks}__{ports} * pd__{banks}__{ports}

<IPython.core.display.Markdown object>

FRML <SUM=TOTAL> EL__ALPHA__RETAIL = EXPOSURE__ALPHA__RETAIL*PD__ALPHA__RETAIL $
FRML <SUM=TOTAL> EL__ALPHA__SME = EXPOSURE__ALPHA__SME*PD__ALPHA__SME $
FRML <SUM=TOTAL> EL__BETA__RETAIL = EXPOSURE__BETA__RETAIL*PD__BETA__RETAIL $
FRML <SUM=TOTAL> EL__BETA__SME = EXPOSURE__BETA__SME*PD__BETA__SME $
FRML <SUM=TOTAL> EL__TOTAL = ((EL__ALPHA__RETAIL+EL__ALPHA__SME)+(EL__BETA__RETAIL+EL__BETA__S
```

Now combine all segments into the full model by calling `%%Makemymodel` (or `%Makemymodel` as a line magic) **without** the `segment` option:

```
%%Makemymodel stress show draw
## Full stress test model
Combining all segments into one model.

<IPython.core.display.Markdown object>

FRML <> PD__ALPHA__RETAIL = 0.02+0.8*PD__ALPHA__RETAIL( -1)+0.1*GDP_GROWTH $
FRML <> PD__ALPHA__SME = 0.02+0.8*PD__ALPHA__SME( -1)+0.1*GDP_GROWTH $
FRML <> PD__BETA__RETAIL = 0.02+0.8*PD__BETA__RETAIL( -1)+0.1*GDP_GROWTH $
FRML <> PD__BETA__SME = 0.02+0.8*PD__BETA__SME( -1)+0.1*GDP_GROWTH $
FRML <SUM=TOTAL> EL__ALPHA__RETAIL = EXPOSURE__ALPHA__RETAIL*PD__ALPHA__RETAIL $
FRML <SUM=TOTAL> EL__ALPHA__SME = EXPOSURE__ALPHA__SME*PD__ALPHA__SME $
FRML <SUM=TOTAL> EL__BETA__RETAIL = EXPOSURE__BETA__RETAIL*PD__BETA__RETAIL $
FRML <SUM=TOTAL> EL__BETA__SME = EXPOSURE__BETA__SME*PD__BETA__SME $
FRML <SUM=TOTAL> EL__TOTAL = ((EL__ALPHA__RETAIL+EL__ALPHA__SME)+(EL__BETA__RETAIL+EL__BETA__S
```

0.3.5 5. Using `replacements=` for dimension placeholders

When variable names have many dimensions, you can define a replacement tuple in the notebook and pass it by name:

```
repl = ('__d', '__{banks}__{ports}')

%%Makemymodel compact show replacements=repl
>list banks = banks : A B
>list ports = ports : X Y
Compact notation using `__d` as placeholder:
>doable <> loss__d = exp__d * pd__d * lgd__d

<IPython.core.display.Markdown object>

FRML <> LOSS__A__X = EXP__A__X*PD__A__X*LGD__A__X $
FRML <> LOSS__A__Y = EXP__A__Y*PD__A__Y*LGD__A__Y $
FRML <> LOSS__B__X = EXP__B__X*PD__B__X*LGD__B__X $
FRML <> LOSS__B__Y = EXP__B__Y*PD__B__Y*LGD__B__Y $
```

0.3.6 7. Tags inside the magic

All `Makemodel` tags work inside the magic cells. Here is a quick reference:

- `<exo>` — allow fixing the variable to an exogenous value
- `<add>` — generate add-factor adjustment variables
- `<fit>` — compute the original fitted value alongside the adjusted one
- `<sum=label>` — inside `doable`, also generate an aggregation equation
- `<add_suffix=__J>` — change the default add-factor suffix from `_A`

Tags are placed between `><` and `>` at the start of a formula line:

```
%%Makemymodel tagged show
A fully decorated equation:
><exo,add,fit> consumption = 0.8 * income
```

```
<IPython.core.display.Markdown object>
```

```
FRML <EXO,ADD,FIT> CONSUMPTION = (0.8*INCOME + CONSUMPTION_A)* (1 -CONSUMPTION_D)+ CONSUMPTION
FRML <FIT>  CONSUMPTION_FITTED = 0.8*INCOME $
FRML <CALC_ADD_FACTOR>  CONSUMPTION_A = (CONSUMPTION) - (0.8*INCOME) $
```

0.3.7 8. Generating a LaTeX PDF

With the `latex` option, the magic converts the cell's Markdown text (headings, tables, bullet lists, equation lines) into LaTeX and compiles a PDF. This requires a LaTeX installation on the system.

```
%%Makemymodel mymodel latex
# My Model Documentation
> y = c + i
```

0.3.8 9. How it works internally

The magic is implemented in `modeljupytermagic.py`. Here is the flow:

1. **Parse the first line** — `get_options(line)` splits it into the model name and a dictionary of options.
2. **Handle segments** — if `segment=` is present, the cell text is stored in `<name>_dict[segment]`. List/text segments are rendered and returned early. Other segments combine list definitions with the current cell.
3. **Build the model** — the combined text is passed to `Makemodel(model_text, replacements=..., funks=...)`. This does all DSL expansion: lists, loops, doable, sums, tags.
4. **Push to namespace** — the resulting `Makemodel` instance is injected into the notebook's user namespace via `ip.push()`.
5. **Render output** — depending on options, the magic renders Markdown, prints expanded equations (`.show`), draws the dependency graph (`.draw`), or generates LaTeX.

The key insight is that the magic is a **thin wrapper** — all the heavy lifting is done by `Makemodel`. The magic simply provides a convenient notebook-native interface for defining and documenting models in one place.

0.3.9 10. Quick reference

```
%%Makemymodel <name> [show] [draw] [display] [render=0] [latex] [segment=<seg>] [replacements=
<Markdown text with > prefixed DSL equations>
```

After execution: the variable `<name>` holds the `Makemodel` instance with all the usual properties — `.show`, `.draw`, `.mmodel`, `.render`, etc.

0.4 The normal function

The function `normal` from the module `modelnormalize` **normalizes** model equations: it takes a business-logic equation written as `lhs = rhs` and rewrites it so that the equation's endogenous variable stands alone on the left-hand side. In a model each equation determines one endogenous variable, so the normalized form of equation i is $y_i = F_i(y, x)$ — the right-hand side may contain the other endogenous variables of the model and any lagged (or leaded) values, including lags of y_i itself; only the current-period y_i must not appear on the right. This is the form the ModelFlow solvers require.

Beyond isolating the endogenous variable, `normal` can:

- **preprocess** EViews-style transformation functions (`DLOG`, `DIFF`, `PCT_GROWTH`, `PCY`, `MOVAVG`, `LOGIT`, `D`, ...) into elementary expressions,
- **invert** a transformation applied to the endogenous variable on the left-hand side (for instance solve `dlog(y) = rhs` for `y`),
- inject an **add factor** (adjustment term) `<var>_A` together with the equation that calculates it from data,
- make the equation **fixable** (exogenizable) through a dummy `<var>_D` and a fixing value `<var>_X`,
- produce a **fitted** version of the equation, stripped of add factor and fixing terms,
- recast **implicit** equations (equilibrium conditions) as residual equations for the solver.

`normal` is the workhorse behind `Makemodel` and therefore also the `%%Makemymodel` cell magic. It returns an instance of the class `Normalized_frml`, which holds the different variants of the equation.

This notebook walks through the main use cases. The reader is assumed to be comfortable with Python and with macroeconomic model equations.

0.4.1 Imports

Import the `normal` function and the `Normalized_frml` result class.

```
from modelnormalize import normal,Normalized_frml
```

0.4.2 The result class: `Normalized_frml`

Every call to `normal` returns a `Normalized_frml` instance. Its attributes hold the different representations of the equation — original, preprocessed, normalized, the add-factor calculation, the fitted version and so on. Displaying the instance prints the attributes that are set.

```
print(Normalized_frml.__doc__)
```

```
Class defining the result from normalization of an expression.

Attributes:
    endo_var (str): The endogenous variable.
    eviews (str): The EViews representation of the expression.
    original (str): The original Business logic expression.
    preprocessed (str): The preprocessed expression.
    normalized (str): The normalized expression.
    calc_add_factor (str): The calculated additional factor.
    un_normalized (str): The un -normalized expression.
    fitted (str): The fitted expression.
```

0.4.3 The signature of `normal`

The docstring below lists all arguments. The ones used in this notebook are `the_endo`, `add_add_factor`, `add_suffix`, `endo_lhs`, `make_fixable`, `make_fitted` and `implicit`.

```
print(normal.__doc__)
```

```
normalize an expression g(y,x) = f(y,x) ==> y = F(x,z)

Default find the expression for the first variable on the left hand side (lhs)

The variable - without lags -  should not be on rhs.

Args:
    ind_o (str): input expression, no $ and no frml name just lhs=rhs
    the_endo (str, optional): the endogeneous to isolate on the left hans side. if the first v
                              It shoud be on the left hand side.
    add_add_factor (bool, optional): force introduction aof adjustment term, and an expression
    do_preprocess (bool, optional): DESCRIPTION. preprocess the expression
    endo_lhs (bool, optional): If false, accept to normalize for a rhs endogeneous variable
    make_fixable  (bool, optional): also make this equation exogenizable
    fitted (bool,optional) : create a fitted equations, without exo and adjustment
    implicit (bool,optional) : This is an implicit frml so to transform to endo__res = (lhs) -

preprocessing handels


Returns:
     An instance of the class: Normalized_frml which will contain the different relevant expre
```

Autoreload keeps the imported module in sync with the source file — convenient while developing `modelnormalize`, not needed if you just use the function.

0.4.4 Basic normalization

The simplest possible case: `a = b`. Variable names are upper-cased, and by default an **add factor** `A_A` is introduced. `Calc_add_factor` shows the expression used to compute the add factor from historical data, so that the equation tracks the data exactly.

```
normal('a=b')

Endo_var        : A
Original        : a=b
Preprocessed    : A=B
Normalized      : A = (B + A_A)
Calc_add_factor : A_A = (A) - (B)
```

With `add_add_factor=0` no adjustment term is introduced — the result is the plain normalized equation.

```
normal('a=b',add_add_factor=0)

Endo_var        : A
Original        : a=b
Preprocessed    : A=B
Normalized      : A = B

normal('a -c=b',add_add_factor=0)

Endo_var        : A
Original        : a -c=b
Preprocessed    : A -C=B
Normalized      : A = C+ (B)
```

0.4.5 Add factor, fixing and fitted equation combined

All the bells and whistles at once:

- `add_suffix='_AERR'` gives the add factor a custom name (`A_AERR` instead of the default `A_A`),
- `make_fixable=True` makes the equation exogenizable: the dummy `A_D` and the fixing value `A_X` are introduced, so setting `A_D = 1` fixes `A` at the value of `A_X`,
- `make_fitted=True` creates the fitted equation `A_FITTED` — the pure right-hand side without add factor and fixing terms.

```
normal('a=b',add_add_factor=1,add_suffix= '_AERR',make_fitted=True,make_fixable=True )
```

```
Endo_var        : A
Original        : a=b
Preprocessed    : A=B
Normalized      : A = (B + A_AERR)* (1 -A_D)+ A_X*A_D
Calc_add_factor : A_AERR = (A) - (B)
Fitted          : A_FITTED = B
```

0.4.6 Built-in transformation functions on the right-hand side

The preprocessing step unfolds EViews-style transformation functions into elementary expressions. First `dlog(b)` — the change in the logarithm — which becomes `log(B) - log(B(-1))`.

```
normal('a=dlog(b)',add_add_factor=1,add_suffix= '_AERR',make_fitted=True,make_fixable=True )
```

```
Endo_var        : A
Original        : a=dlog(b)
Preprocessed    : A=((LOG(B)) -(LOG(B( -1))))
Normalized      : A = (((log(B)) -(log(B( -1)))) + A_AERR)* (1 -A_D)+ A_X*A_D
Calc_add_factor : A_AERR = (A) - (((log(B)) -(log(B( -1)))))
Fitted          : A_FITTED = ((LOG(B)) -(LOG(B( -1))))
```

`logit(b)` is unfolded to `log(B/(1 - B))` — useful when a variable must stay between 0 and 1.

```
normal('a=logit(b)',add_add_factor=1,add_suffix= '_AERR',make_fitted=True,make_fixable=True )
```

```
Endo_var        : A
Original        : a=logit(b)
Preprocessed    : A=(LOG(B/(1.0 -B)))
Normalized      : A = ((log(B/(1.0 -B))) + A_AERR)* (1 -A_D)+ A_X*A_D
Calc_add_factor : A_AERR = (A) - ((log(B/(1.0 -B))))
Fitted          : A_FITTED = (LOG(B/(1.0 -B)))
```

`PCT_GROWTH(b)` is the percentage growth from the previous period: `100*(B/B(-1) - 1)`.

```
normal('a=PCT_GROWTH(b)',add_add_factor=0)
```

```
Endo_var        : A
Original        : a=PCT_GROWTH(b)
Preprocessed    : A= (100 * ( (B) / (B( -1)) -1))
Normalized      : A =  (100 * ( (B) / (B( -1)) -1))
```

`PCY(b)` is the year-over-year percentage growth — the change over four periods, so it assumes quarterly data.

```
normal('a=PCY(b)',add_add_factor=1,add_suffix= '_AERR',make_fitted=True,make_fixable=True )
```

```
Endo_var        : A
Original        : a=PCY(b)
Preprocessed    : A= (100 * ( (B) / (B( -4)) -1))
Normalized      : A = ( (100 * ( (B) / (B( -4)) -1)) + A_AERR)* (1 -A_D)+ A_X*A_D
Calc_add_factor : A_AERR = (A) - ( (100 * ( (B) / (B( -4)) -1)))
Fitted          : A_FITTED =  (100 * ( (B) / (B( -4)) -1))
```

`DIFF(b)` is the first difference `B - B(-1)`.

```
normal('a=DIFF(b)',add_add_factor=1,add_suffix= '_AERR',make_fitted=True,make_fixable=True )
```

```
Endo_var        : A
Original        : a=DIFF(b)
Preprocessed    : A=((B) -(B( -1)))
Normalized      : A = (((B) -(B( -1))) + A_AERR)* (1 -A_D)+ A_X*A_D
Calc_add_factor : A_AERR = (A) - (((B) -(B( -1))))
Fitted          : A_FITTED = ((B) -(B( -1)))
```

`MOVAVG(b,3)` is the moving average over the current and the two preceding periods.

```
normal('a=MOVAVG(b,3)',add_add_factor=1,add_suffix= '_AERR',make_fitted=True,make_fixable=True
```

```
Endo_var        : A
Original        : a=MOVAVG(b,3)
Preprocessed    : A=((B+B( -1)+B( -2))/3.0)
Normalized      : A = (((B+B( -1)+B( -2))/3.0) + A_AERR)* (1 -A_D)+ A_X*A_D
Calc_add_factor : A_AERR = (A) - (((B+B( -1)+B( -2))/3.0))
Fitted          : A_FITTED = ((B+B( -1)+B( -2))/3.0)
```

0.4.7 Transformations on the left-hand side

When the endogenous variable enters through a transformation on the left-hand side, `normal` **inverts** the transformation. Here `PCT_growth(a) = n(-1)` is solved for the level of `A`.

```
normal('PCT_growth(a) = n( -1)',add_add_factor=0)
```

```
Endo_var        : A
Original        : PCT_growth(a) = n( -1)
Preprocessed    :  (100 * ( (A) / (A( -1)) -1)) =N( -1)
Normalized      : A =  (N( -1)) *A( -1)/100+A( -1)
```

Transformation functions can be nested — here a moving average of the percentage change `pct(b)`. Note how the lag operator is distributed over the arguments when the expression is unfolded.

```
normal('a = movavg(pct(b),2)',add_add_factor=0)
```

```
Endo_var        : A
Original        : a = movavg(pct(b),2)
Preprocessed    : A=(( (100 * ( (B) / (B( -1)) -1)) + (100 * ( (B( -1)) / (B( -2)) -1)) )/2.0)
Normalized      : A = (( (100 * ( (B) / (B( -1)) -1)) + (100 * ( (B( -1)) / (B( -2)) -1)) )/2.
```

`pct_growth` on **both** sides: the left-hand side is inverted to isolate `C`, while the right-hand side is simply unfolded.

Note: the right-hand variable is called `DD` because `D` is reserved for the EViews difference operator `D(...)` — once the equation is unfolded, a lag like `D(-1)` would be indistinguishable from a call to the operator, so `normal` raises an error asking you to rename such a variable.

```
normal('pct_growth(c) = pct_growth(dd)',add_add_factor=0)
```

```
Endo_var        : C
Original        : pct_growth(c) = pct_growth(dd)
Preprocessed    :  (100 * ( (C) / (C( -1)) -1)) = (100 * ( (DD) / (DD( -1)) -1))
Normalized      : C =  ((100 * ( (DD) / (DD( -1)) -1))) *C( -1)/100+C( -1)
```

A variable can not share its name with one of the reserved function names (`D`, `DIFF`, `DLOG`, `PCT_GROWTH`, `PCY`, `LOGIT`, `MOVAVG`): as soon as such a variable appears with a lag — like `DIFF(-1)` — it is indistinguishable from a function call, and `normal` raises an explicit error asking you to rename it. In the two cells below the exception is caught just to display the message.

First a variable named `DIFF`, lagged directly by the user:

```
try:
    normal('a = diff(b) + diff( -1)',add_add_factor=0)
except Exception as e:
    print('Exception raised')
    print(e)
```

```
Exception raised
A variable can not be named DIFF, as DIFF( is a function in the business language.
DIFF( -1) looks like a lagged variable DIFF. Rename the variable.
Original expression: a=diff(b)+diff( -1)
Processed so far   : A=((B) -(B( -1)))+DIFF( -1)
```

And a variable named `D` — here the offending lag `D(-1)` is not written by the user but *created* by the unfolding of `pct_growth(d)`, as the "Processed so far" line shows:

```
try:
    normal('pct_growth(c) = pct_growth(d)',add_add_factor=0)
except Exception as e:
    print('Exception raised')
    print(e)
```

```
Exception raised
A variable can not be named D, as D( is a function in the business language.
D( -1) looks like a lagged variable D. Rename the variable.
Original expression: pct_growth(c)=pct_growth(d)
Processed so far   :  (100 * ( (C) / (C( -1)) -1)) = (100 * ( (D) / (D( -1)) -1))
```

When the left-hand side is a transformation and an add factor is requested (the default), the add factor enters in the **transformed space**: here `C_A` is measured in percentage growth, and `Calc_add_factor` computes it accordingly.

```
normal('pct_growth(c) = z+pct(b) + pct(e)')
```

```
Endo_var        : C
Original        : pct_growth(c) = z+pct(b) + pct(e)
Preprocessed    :  (100 * ( (C) / (C( -1)) -1)) =Z+ (100 * ( (B) / (B( -1)) -1)) + (100 * ( (E
Normalized      : C = C_A*C( -1)/100+ (Z+ (100 * ( (B) / (B( -1)) -1)) + (100 * ( (E) / (E( -1
Calc_add_factor : C_A = 100*C/C( -1) - ((Z+ (100 * ( (B) / (B( -1)) -1)) + (100 * ( (E) / (E(
```

0.4.8 A real-world example

An estimated error-correction equation from a World Bank EViews model. `DLOG` on the left-hand side gives a **multiplicative** normalization through `EXP(...)`, and the add factor works on the log change. EViews-specific pieces such as `@DURING("2011")` pass through the preprocessing. The `.fprint` attribute prints the result without truncation — handy for long equations.

```
normal("DLOG(SAUNECONGOVTXN) = -0.323583422052*(LOG(SAUNECONGOVTXN( -1)) -GOVSHAREWB*LOG(SAUNE
```

```
Endo_var        : SAUNECONGOVTXN
Original        : DLOG(SAUNECONGOVTXN) = -0.323583422052*(LOG(SAUNECONGOVTXN( -1)) -GOVSHAREWB
Preprocessed    : ((LOG(SAUNECONGOVTXN)) -(LOG(SAUNECONGOVTXN( -1))))= -0.323583422052*(LOG(SA
Normalized      : SAUNECONGOVTXN = SAUNECONGOVTXN( -1)*EXP(SAUNECONGOVTXN_A+ ( -0.323583422052
Calc_add_factor : SAUNECONGOVTXN_A = - (( -0.323583422052*(LOG(SAUNECONGOVTXN( -1)) -GOVSHAREW
```

`DIFF` on the left-hand side is inverted additively: `A = rhs + A(-1)`.

```
normal("Diff(a) = b")
```

```
Endo_var        : A
Original        : Diff(a) = b
Preprocessed    : ((A) -(A( -1)))=B
Normalized      : A = A_A+ (B) +A( -1)
Calc_add_factor : A_A = A - ((B)) -A( -1)
```

0.4.9 Leads and the EViews `D()` operator

The EViews difference operator `D(x, n, s)` is also handled, and expressions may contain **leads** such as `QLHP(+1)`.

```
normal('a = D( LOG(QLHP(+1)), 0, 1 )')
```

```
Endo_var        : A
Original        : a = D( LOG(QLHP(+1)), 0, 1 )
Preprocessed    : A=((LOG(QLHP(+1))) -(LOG(QLHP)))
Normalized      : A = (((log(QLHP(+1))) -(log(QLHP))) + A_A)
Calc_add_factor : A_A = (A) - (((log(QLHP(+1))) -(log(QLHP))))
```

`D(x)` with default arguments is a plain first difference — the result is the same as above.

```
normal('a = D( LOG(QLHP(+1)))')
```

```
Endo_var        : A
Original        : a = D( LOG(QLHP(+1)))
Preprocessed    : A=((LOG(QLHP(+1))) -(LOG(QLHP)))
Normalized      : A = (((log(QLHP(+1))) -(log(QLHP))) + A_A)
Calc_add_factor : A_A = (A) - (((log(QLHP(+1))) -(log(QLHP))))
```

0.4.10 Choosing the endogenous variable

By default the first variable on the left-hand side becomes the endogenous variable. With `the_endo` and `endo_lhs=False` the equation can instead be solved for a variable on the **right-hand side** — here `F`. Combined with `make_fixable=True` the resulting equation is also exogenizable.

```
normal('a = gamma+ f+0',the_endo='f',endo_lhs=False,make_fixable =True)
```

```
Endo_var        : F
Original        : a = gamma+ f+0
Preprocessed    : A=GAMMA+F+0
Normalized      : F = (A -F_A -GAMMA -0) * (1 -F_D)+ F_X*F_D
Calc_add_factor : F_A = A -F -GAMMA -0
```

0.4.11 Implicit equations

Some equations — typically equilibrium conditions — cannot be solved analytically for the endogenous variable. With `implicit=True` the equation is recast as a **residual equation**: `PRICE___RES = supply - demand`, which is driven to zero by adjusting the variable named in `the_endo` (here `PRICE`).

Note that a model containing implicit equations has to be solved with one of the **Newton-type solvers** — the Gauss-Seidel type solvers require every equation to be normalized with the endogenous variable alone on the left-hand side, which is exactly what an implicit equation lacks.

```
normal('demand = supply',the_endo='price',implicit=True)

Endo_var        : PRICE
Original        : demand = supply
Preprocessed    : DEMAND=SUPPLY
Normalized      : PRICE___RES = ( SUPPLY ) - ( DEMAND )
Un_normalized   : PRICE___RES = ( SUPPLY ) - ( DEMAND )
```

0.4.12 Summary: the preprocessing steps

The table lists the preprocessing steps in the order they are applied. Each rewrite is repeated until no occurrence is left, so the functions can be nested — `movavg(pct(b),2)` for instance is unfolded completely.

Step	Pattern	Rewritten as	Meaning
1	*whole equation*	upper-cased, blanks removed	housekeeping
2	`PCT(x)`	`PCT_GROWTH(x)`	alias — handled in step 6
3	`DLOG(x)`	`DIFF(LOG(x))`	change in the logarithm — handled further in step 8
4	`LOGIT(x)`	`(LOG(x/(1.0 - x)))`	log-odds of a variable between 0 and 1
5	`MOVAVG(x,n)`	`((x + x(-1) + ...+ x(-n+1))/n)`	moving average over `n` periods
6	`PCT_GROWTH(x)`	`(100*((x)/(x(-1)) - 1))`	percentage growth from the previous period
7	`PCY(x)`	`(100*((x)/(x(-4)) - 1))`	year-over-year percentage growth (quarterly data)
8	`DIFF(x)`	`((x) - (x(-1)))`	first difference
9	`D(x)` or `D(x,0,1)`	`((x) - (x(-1)))`	EViews difference operator

Before every rewrite the argument is checked: a pure signed integer like `DIFF(-1)` must be a *lagged variable* named like a function, and an explicit error is raised (see the section on reserved names above).

0.5 When using Makemodel og %%Makemymodel the tag <add>will introduce an add factor

In this we will look into some of the behavior of this

0.5.1 Activate the %%Makemymodel magic cell

This is a convinient wrapper around the Makemodel function

```
import modeljupytermagic # ti activate the %% %%Makemymodel magic cell
```

0.5.2 A simple equation

When the equation is already normalized

That is the left hand side is a variable

```
%%Makemymodel test show
> a = b

<IPython.core.display.Markdown object>

FRML <> A = B $

%%Makemymodel test show
><add> a = b

<IPython.core.display.Markdown object>

FRML <ADD> A = (B + A_A)                                $

FRML <CALC_ADD_FACTOR>  A_A = (A) - (B) $

%%Makemymodel test show
><add> a = log(b)

<IPython.core.display.Markdown object>

FRML <ADD> A = (log(B) + A_A)                                    $

FRML <CALC_ADD_FACTOR>  A_A = (A) - (log(B)) $
```

When the equation has to be normalized no differences on left hand side

That is the left hand side is a variable

```
%%Makemymodel test show
><add> log(a) = b

<IPython.core.display.Markdown object>

FRML <ADD> A = EXP(A_A+ (B) ) $

FRML <CALC_ADD_FACTOR>  A_A = - ((B)) +LOG(A) $

%%Makemymodel test show
><add> log(a) = diff(b)
```

```
<IPython.core.display.Markdown object>

FRML <ADD> A = EXP(A_A+ (((B) -(B( -1)))) ) $

FRML <CALC_ADD_FACTOR>  A_A = - ((((B) -(B( -1))))) +LOG(A) $

%%Makemymodel test show
><add> log(a)+d+e = b

<IPython.core.display.Markdown object>

FRML <ADD> A = EXP(A_A -D -E+ (B) ) $

FRML <CALC_ADD_FACTOR>  A_A = D+E - ((B)) +LOG(A) $
```

When the equation has to be normalized have differences on left hand side

That is the left hand side is a variable

```
%%Makemymodel test show
><add> diff(a) = b

<IPython.core.display.Markdown object>

FRML <ADD> A = A_A+ (B) +A( -1) $

FRML <CALC_ADD_FACTOR>  A_A = A - ((B)) -A( -1) $

%%Makemymodel test show
><add, exo,fit> dlog(a) = b

<IPython.core.display.Markdown object>

FRML <ADD, EXO,FIT> A = (A( -1)*EXP(A_A+ (B) )) * (1 -A_D)+ A_X*A_D  $
FRML <FIT>  A_FITTED = A( -1)*EXP( (B) ) $
FRML <CALC_ADD_FACTOR>  A_A = - ((B)) +LOG(A) -LOG(A( -1)) $

%%Makemymodel test show
><add, exo,fit> a = b *c

<IPython.core.display.Markdown object>

FRML <ADD, EXO,FIT> A = (B*C + A_A)* (1 -A_D)+ A_X*A_D  $
FRML <FIT>  A_FITTED = B*C $
FRML <CALC_ADD_FACTOR>  A_A = (A) - (B*C) $
```